
Homework #2 - Multi-Level Indexing

Due Date : 17.05.2026 23.55

Objective

The goal of this assignment is to design and implement a **multi-level secondary indexing system** for a structured retail dataset. Students will develop a three-level index hierarchy **Country, City, and Product** to obtain high-performance data retrieval. The project shows the distinction between physical storage and logical pointer traversal, requiring students to build a system that can access specific records directly without performing a full linear scan of the underlying data files.

Background & Terminology

Data Storage

- Convert the given JSON dataset into a binary data file (.dat) using appropriate **struct** definitions in C.
- Each product record must be stored in the data file and uniquely identifiable (via **product_id**).

Index Construction

Build a **three-level secondary index** consisting of:

- **Level 1:** Country index
- **Level 2:** City index (within each country)
- **Level 3:** Product index (within each city)
- **Level 4:** Actual .dat file

Sorting Strategy

- Apply **Replacement Selection Sort** to create sorted runs for:
 - Countries (Level 1)
 - Cities (Level 2)
 - Products (Level 3)

Linked Index Structure

- Each level must be connected using **linked-list offset pointers**:
 - Country entries point to their first city
 - City entries point to their products in **alphabetical order**
 - Product entries point to the corresponding record in the .dat file

Access Mechanism

- Implement traversal logic such that a query follows:
Country → City → Product → Data File Record

- Ensure that access is performed using indexes only (no full file scan).

Index Structure — Example

Country Index (sorted — Replacement Selection Sort)

Country	Offset → City Index
Germany	1
Italy	3
Turkiye	5

Note: Sorted alphabetically A→Z via Replacement Selection Sort. Offset points to the first city row of that country in the City Index.

City Index (insertion order — linked in alphabetical order per country Replacement Selection Sort)

#	City	Next City	Product Index
[1]	Berlin	2	1
[2]	Hamburg	-1 (end)	3
[3]	Rome	6	4
[4]	Istanbul	-1 (end)	5
[5]	Ankara	4	7
[6]	Venice	-1 (end)	6

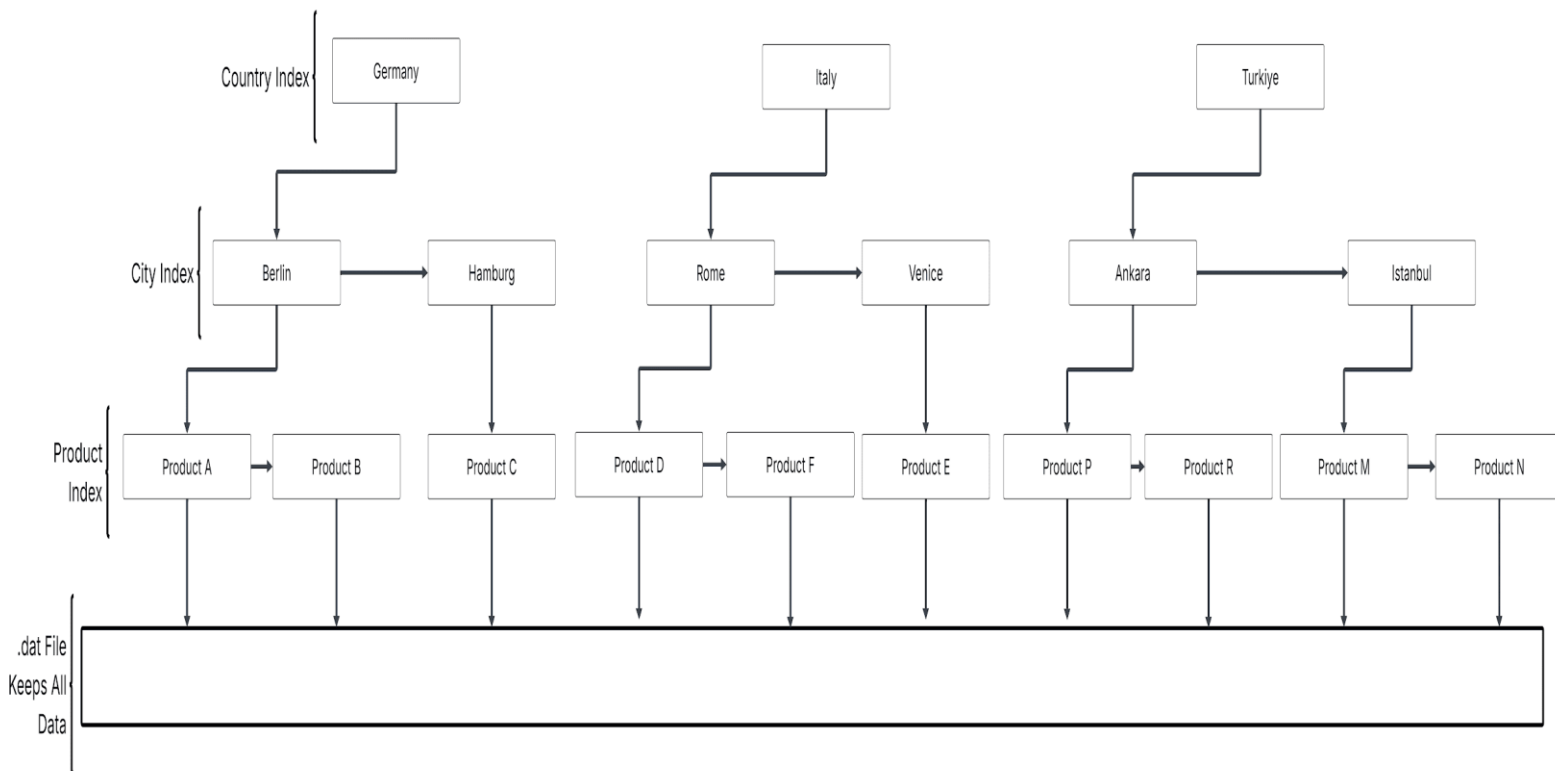
Product Index (sorted alphabetically per city with Replacement Selection Sort)

#	Product Name	Product Offset	.dat Offset
[1]	Dyson V15 Detect	2	rec 1
[2]	Samsung Galaxy S24	-1	rec 2
[3]	Apple MacBook Air M3	-1	rec 3
[4]	DeLonghi Magnifica S	-1	rec 4
[5]	Lenovo Legion Slim 5	6	rec 5
[6]	Philips Airfryer XXL	-1	rec 6
[7]	Xiaomi Mi Band 8	-1	rec 7

The tables illustrate a logical path: **Germany** points to its first city, **Berlin**. **Berlin** then serves as an entry point to the **Product Index**, pointing to its first available product. To find

additional products in Berlin, the system simply follows the 'Next Product' pointers within that index.

Example Indexing



Understanding the Index Structure

- **The Country Level :** Countries are stored side-by-side in memory. This means they are physically sorted via replacement selection sort, allowing the system to locate a specific country quickly using its known position.
- **The City & Product Levels :** Unlike countries, these indexes do not have to stay in order within the physical memory. They can be scattered anywhere. **However, they must be accessed via pointers** and also **these indexes must be sorted with replacement selection sort** in ascending order.

Example

When you are at a Product (like Product A), the "Offset" pointer must always point to the next item in **ascending alphabetical order**. **If you add a new product**, it might be stored at the very end of your memory file physically, but its pointer will correctly link it between the product that come before and after it alphabetically. For efficiency, physically moving data every time you add a new city or product is expensive for the computer. By using pointers, we keep the data alphabetically sorted for the user without having to reorganize the actual memory every time a change is made.

Tasks

Task 1 (20 pts pts)

Using the provided JSON dataset, implement Replacement Selection Sort to produce the sorted Country Index.

Task 2 (25 pts pts)

Build the City Index by inserting cities. For each country, compute the 'Next City' offset values that form an alphabetically ordered linked list. Fill in the complete City Index table with all offset values, including -1 sentinels.

Task 3 (25 pts pts)

Build the Product Index for every city. Products must appear in alphabetical order within each city group. Fill in the 'Next Product' offset column (or -1 if last) and assign a '→.dat offset' pointing to the corresponding record in the .dat file.

Task 4 (20 pts pts)

Trace the full lookup path for the query: For example "List all products available in Turkiye, alphabetically by city and then by product name." Write out each index table access step: row number accessed, field read, pointer followed, and the .dat record retrieved.

Task 5 (10 pts pts)

Upon adding new product to specific cities, you must re-calculate the offsets and next' pointers to ensure the entire hierarchy remains logically sorted and traversable.

Example : Inserting "Product A" into Berlin

In this example, Product D and Product E are already in the system. We are adding Product A. Note how its physical location (Row 3) is at the end, but the pointers reroute the path to make it the first item logically.

Phase 1: Before Adding Product A

Berlin -> Product D -> Product E (Before adding Product A)

Row #	Product Name	Next Product Pointer	.dat Offset
[1]	Product D	2 (Points to E)	rec_d
[2]	Product E	-1 (End of List)	rec_e

Phase 2: After Adding Product A

We append Product A to the end of the file (Row 3). We do not move D or E. Instead, we update the "Entry Point" from the City Index and the pointers in the Product Index.

City Index (Update)

- Berlin's Product Offset changes from 1 to 3.

Product Index Table (Updated File View)

Row #	Product Name	Next Product Pointer	.dat Offset
[1]	Product D	2 (Points to E)	rec_d
[2]	Product E	-1 (End of List)	rec_e
[3]	Product A	1 (Points to D)	rec_a

Berlin -> Product A -> Product D -> Product E (After adding Product A)

Interactive Console UI Requirement

In addition to completing the tasks above, students must implement an **interactive console-based user interface** in C that allows the user to interact with the multi-level index system at runtime. The program must not terminate after a single operation — it should continuously present a menu and await user input until the user chooses to exit.

Required Console Menu Options:

- **Search by Country/City/Product:** The user must be able to enter a country name, a city name, or a product name and retrieve the corresponding records by traversing the index hierarchy. No full file scan is permitted.
- **Sort and Display Index Levels:** The user must be able to choose any index level (Country, City, or Product) and display its contents in sorted order, demonstrating that the linked-list pointers maintain correct alphabetical traversal.
- **Insert a New Product:** The user must be able to add a new product to a specified city, after which the program must automatically recalculate and update all affected offset and next-pointer values across the index hierarchy.
- **Apply Replacement Selection Sort:** The console must include an option to trigger and display the step-by-step execution of Replacement Selection Sort on the Country Index, showing each run produced.
- **Exit:** Cleanly close all open file handles and free allocated memory before terminating the program.

Note: All search and sort operations performed through the console must operate exclusively through the index structures. Direct linear scanning of the .dat file is not permitted and will result in point deductions.

Submission Guidelines

- Name your C files as **<your_id>.c**
- Ensure readable, documented and well structured code.
- Late submission will not be accepted.
- If you fail to follow the naming conventions you will be deducted **10 pts**.
- Your submissions will be scanned among each other as well as against internet repositories. Any assignments that exceed the similarity threshold will receive a zero.
- We strongly encourage you not to submit your assignment rather than make a dishonest submission.